



Islamic University of Technology (IUT)

A subsidiary organ of the Organisation of Islamic Cooperation (OIC)

Board Bazar, Gazipur, Bangladesh

TECHNICAL REPORT

HerWILL × UAP Safe Social Media Datathon 2026

Stacking Fine-Tuned LLMs for Robust Harmful-Content Detection in Bangla, English and Banglish

IUT_BakeNekos

Team Members

Md. Farhan Ishraq

Didhiti Nahid

Tamim Muhammad Rayeed

4 October 2026

Stacking Fine-Tuned LLMs for Robust Harmful-Content Detection in Bangla, English and Banglish

Md. Farhan Ishraq, Didhiti Nahid and Tamim Muhammad Rayeed
Team IUT_BakeNekos, Islamic University of Technology (IUT), Gazipur, Bangladesh
HerWILL × UAP Safe Social Media Datathon 2026

Abstract—Subtle toxicity, expressed through sarcasm and coded language, is hard to detect, and harder still in low-resource, code-mixed settings. We present the system of team IUT BakeNekos for the HerWILL × UAP Safe Social Media Datathon 2026, which classifies social-media posts in Bangla, English and Banglish as explicitly toxic, subtly toxic or neutral, and which placed second on the private leaderboard (macro F1 0.7554). Four text classifiers, among them Gemma 4 31B fine-tuned with QLoRA as a class-token classifier, are trained on one pinned 5-fold split. A gradient-boosting stacker combines their out-of-fold probabilities with sequential neighbourhood features built under strict leakage rules: only training annotations are used, no post ever sees its own class, and test posts are treated exactly like training posts. Per-class multipliers then tune the decision rule for macro F1. The design generalises robustly: cross-validation predicted the private score to within 0.0003, every stack submission landed within 0.004 of its estimate, and this generalisation helped us rise from third on the public to second on the private leaderboard. We show that the public split is a contiguous, harder slice of the data, on which macro F1 varies from 0.709 to 0.812, which is why selecting by cross-validation rather than by the public leaderboard was decisive. A three-seed ablation identifies Gemma as the key text member; a stack built on Gemma alone scores 0.7582.

Index Terms—harmful-content detection, implicit toxicity, Bangla, code-mixing, large language models, QLoRA, stacked generalization, leakage-safe validation, macro F1

I. INTRODUCTION

Automatic detection of harmful language is hardest where it is least explicit. Offensive and hateful language are easily confused [1], implicit or coded hate escapes keyword-level models [2], and classifiers inherit annotator and dialect biases [3], [4]. These problems grow in low-resource and code-mixed settings: Bangla has comparatively few annotated resources for the task [5], and social-media users freely mix Bangla script, English and romanised Bangla (Banglish) [6].

The HerWILL × UAP Safe Social Media Datathon 2026 targets exactly this setting. Each post is annotated as *explicitly toxic* (class 0), *subtly toxic* (class 1: sarcasm, coded language) or *non-toxic/neutral* (class 2), and submissions are ranked by macro F1 on a hidden 60% of the test set. Only the provided data may be used, and the competition window was about 24 hours.

In such a setting the leaderboard rewards models that generalise, not models that fit the visible public split. Our system was designed around that principle from the first hour:

every model shares one pinned validation split, every stacked prediction is out-of-fold, every feature obeys explicit leakage rules, and every decision was taken on cross-validation rather than on public scores. The result placed second on the private leaderboard, with a private score within 0.0003 of its cross-validation estimate (Fig. 3). Our contributions are:

- a **leakage-safe two-level system** built on a strict *validation contract* (one pinned split and a fixed output format) that let more than fifty independent training runs be stacked without leakage (Sec. IV);
- a **class-token formulation** that turns a 31B decoder LLM into a classifier without a new head, and a comparison of 17 fine-tuned models, from TF-IDF to Gemma 4 31B, with per-class and per-script scores (Sec. VI-A);
- **sequential neighbourhood features** with verifiable safeguards, including a perturbation test and a documented case of a leaking candidate that we caught and discarded (Sec. IV-D);
- **evidence of robust generalisation**: cross-validation tracked every private score to within 0.004, and an analysis shows why the public leaderboard was a misleading guide (Sec. VIII);
- a **three-seed ablation** of the stacker with stacker-family, member-selection and calibration studies, and a discussion of the Bangla–English gap (Secs. VII, IX).

II. RELATED WORK

Harmful and implicit toxicity. Early work separated hate speech from merely offensive language [1]; later benchmarks target implicit and coded hate [2] and show that classifiers encode annotator and dialect bias [3], [4]. Our class 1 is closest to implicit hate.

Bangla and code-mixed NLP. BanglaBERT [7], MuRIL [8] and IndicBERTv2 [9] were pre-trained for Bangla and other Indic languages; Bangla hate speech has been studied with such encoders [5], and code-switched text poses its own difficulties [6].

Multilingual encoders and LLM fine-tuning. We use XLM-R [10], XLM-T [11], mDeBERTa-v3 [12] and multilingual E5 [13] as encoders, and low-rank adaptation [14] on 4-bit quantised weights [15] for Qwen3 [16] and Gemma 4 [17], with prompted baselines served by vLLM [18]; all models use Hugging Face Transformers [19].

Ensembles, leakage and calibration. Stacked generalisation [20] trains a second-level model on out-of-fold predictions; we use gradient-boosted trees [21] and compare implementations [22]–[25]. Pseudo-annotation [26] adds confident unannotated examples. Target leakage, where a feature encodes the class through how it is computed, is a classic failure [27], and F1-optimal decisions differ from the argmax of calibrated probabilities [28].

III. TASK AND DATA

A. Task and metric

Given the text of a post, predict $y \in \{0, 1, 2\}$. The metric is the unweighted mean of per-class F1,

$$F1_{\text{macro}} = \frac{1}{3} \sum_{c=0}^2 \frac{2P_c R_c}{P_c + R_c}, \quad (1)$$

so the neutral class, 12.3% of the training rows, weighs as much as the explicit class, which covers half of them. The test set is split into a public (40%) and a private (60%) part; the private part decides the ranking.

B. Statistics

TABLE I
DATASET STATISTICS, MEASURED ON THE COMPETITION FILES.

Statistic	Value
Training / test posts	47,817 / 11,955
Classes in train (explicit / subtle / neutral)	49.7 / 38.0 / 12.3%
Posts with Bangla script (train / test)	42.2 / 42.0%
Words per post: median, 99th percentile (train)	8, 58
Duplicate groups in train (rows); conflicting	468 (1,100); 55
Test posts with a verbatim training twin	281 (272 unamb.)
Class agreement of near-twins (cosine ≥ 0.99)	84%
id range (train and test together)	0–59,771

Table I lists the main statistics. Posts are short and informal; emojis, repeated punctuation and capitalisation are common and often carry the sarcasm that defines class 1. Annotations are noisy: near-identical posts share a class only 84% of the time, so macro F1 far above 0.8 is implausible. We used this *annotation-noise ceiling* as a sanity check throughout (Sec. IV-D).

C. Posting sequence

The `id` column runs contiguously from 0 to 59,771 across the training and test files, and the test posts occupy what look like random positions within it. Fig. 1 shows that both script and class mix drift in long, smooth stretches along this axis: ids 0–6,000 are almost entirely Bangla, ids 18,000–24,000 almost entirely English. Fig. 2 quantifies the class dependence: consecutive training posts share a class 0.611 of the time against 0.406 under independence, decaying smoothly to 0.557 at distance 10 and 0.501 at 100. We read the `id` as the order in which posts were gathered, from slowly drifting sources or threads.

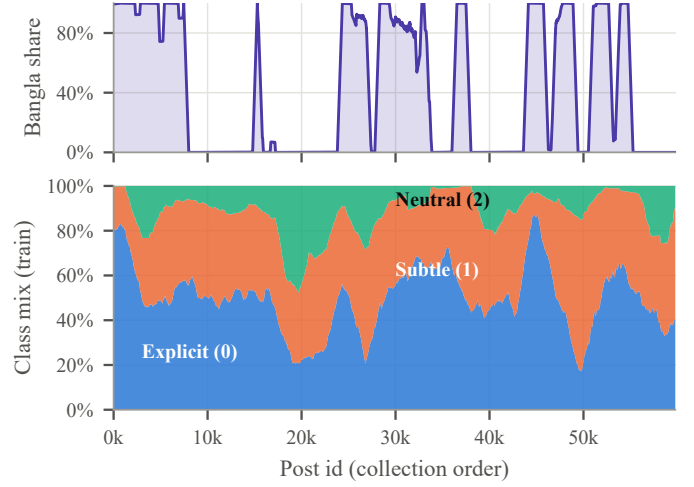


Fig. 1. **The data drift along the id axis.** Top: share of posts written in Bangla script (rolling window of 501 posts, training and test). Bottom: class mix of training posts (rolling window of 2,001 posts). Both change in long, smooth stretches, so a post’s neighbours are informative about its language and its class.

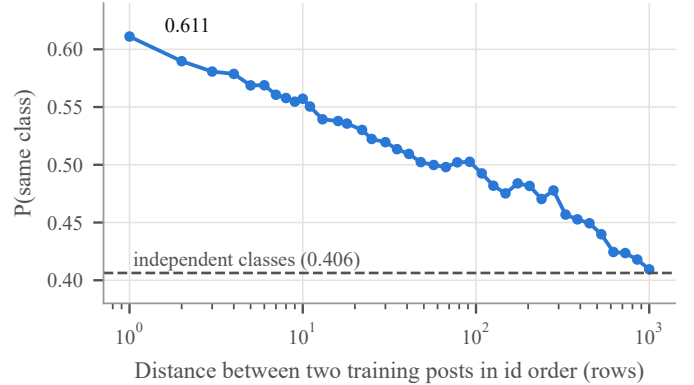


Fig. 2. **Neighbouring posts tend to share a class.** Probability that two training posts carry the same class, against their distance in id order (log scale). The dashed line is the rate expected if classes were independent, $\sum_c \pi_c^2 = 0.406$.

IV. METHODOLOGY

Fig. 3 gives an overview of the pipeline; the subsections below describe each stage.

A. Preprocessing

Preprocessing is deliberately minimal and identical for every model: Unicode NFKC normalisation (Bangla has several encodings of the same glyphs), URLs and @handles replaced by `<url>` and `<user>`, and whitespace collapsed. Emojis, punctuation and casing are kept because they carry much of the sarcasm signal and modern subword tokenizers represent them. There is no stemming, stop-word removal or transliteration. BanglaBERT additionally receives the normaliser it was pre-trained with [7].



Fig. 3. **Overview of the proposed pipeline.** Posts in Bangla, English and Banglish are lightly normalised and scored by four level-1 classifiers trained on one shared stratified 5-fold split. Their out-of-fold class probabilities, together with leakage-safe meta features (Sec. IV-D), feed a HistGradientBoosting stacker bagged over five seeds, and per-class multipliers tune the final decision for macro F1. Exact seed counts, preprocessing steps and feature definitions are given in the following subsections.

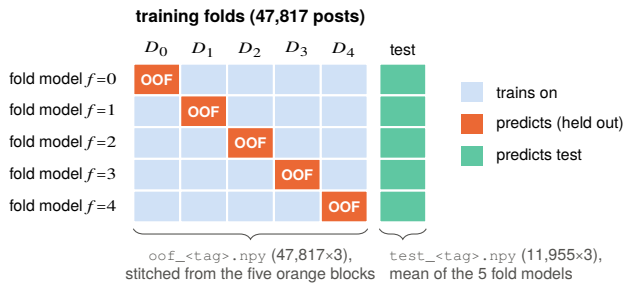


Fig. 4. **The out-of-fold (OOF) contract shared by every run.** Each fold model trains on four folds (blue) and predicts only its held-out fold (orange) and the test set (green). Stitching the held-out blocks gives an OOF matrix in which no row is ever scored by a model that saw it, so any set of runs can be stacked without leakage.

B. Validation contract

All experiments share one split: rows are sorted by id and folds come from `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` [25] (fold sizes 9,564 / 9,564 / 9,563 / 9,563 / 9,563, asserted in every script). Every run, on any machine, must write three artifacts (Fig. 4): `oof_<tag>.numpy` (47,817×3 probabilities, each row predicted by the fold model that did not train on it), `test_<tag>.numpy` (11,955×3, the mean of the five fold models) and `log_<tag>.json` (configuration, per-fold scores, training history). This contract is what makes stacking safe: the level-2 model learns how far to trust each member only from predictions on rows that member never saw.

C. Level-1 text models

TF-IDF. Word 1–2-grams and character 2–5-grams (within word boundaries, sublinear TF) feed a class-balanced logistic regression with $C = 4$.

Encoders. We fine-tune eleven encoders (Table III), each with a linear head on the first token, using AdamW (weight decay 0.01), a cosine schedule with 6% warm-up, three epochs, maximum length 128, bf16 and class-weighted cross-entropy with $w_c = N/(3n_c)$. multilingual-e5-large uses learning rate 10^{-5} and batch 16.

Qwen3-32B with a classification head. Weights are loaded in 4-bit NF4 [15]; LoRA adapters [14] ($r = 16$, $\alpha = 32$, dropout 0.05) are attached to all attention and MLP projections (q, k, v, o, gate, up, down) together with a sequence-classification head that pools the last non-padding token. Training runs one epoch with effective batch 32, learning rate $5 \cdot 10^{-5}$, 3% warm-up, fp32 logits and length-bucketed batches. The Qwen tokenizer splits Bangla into about 2.7 times more tokens than XLM-R, so the maximum length is 192.

Gemma 4 31B as a class-token classifier. Our Transformers version has no sequence-classification class for Gemma 4 [17]. Instead of adding a randomly initialised head, we cast classification as next-token prediction. The prompt asks the model to classify the post (Bangla, English or Banglish), defines the three classes in one line each (0 = explicitly toxic or hateful, 1 = subtly toxic: sarcasm, coded language, 2 = non-toxic or neutral), gives the cleaned post and ends with an answer slot. We take the logits z_0, z_1, z_2 of the tokens “0”, “1”, “2” at the final position as class scores, $p = \text{softmax}(z_0, z_1, z_2)$. Training minimises class-weighted cross-entropy over these three logits; left padding keeps the answer slot at the last position, and the post is truncated so that the suffix always survives. The model reuses its pre-trained output layer, so no parameter starts from scratch. LoRA settings match Qwen; the maximum length is 128 (the Gemma tokenizer’s 99th percentile is 77 tokens). Table II lists the final members’ settings.

TABLE II
SETTINGS OF THE FOUR FINAL LEVEL-1 MEMBERS.

	Gemma 4 31B	Qwen3-32B	mE5-large	TF-IDF
Training	QLoRA	QLoRA	full FT	convex
Head	class tokens	cls. head	linear	LogReg
Quantisation	NF4	NF4	—	—
LoRA $r / \alpha / \text{drop}$.	16 / 32 / .05	16 / 32 / .05	—	—
Max. tokens	128	192	128	—
Learning rate	$5 \cdot 10^{-5}$	$5 \cdot 10^{-5}$	10^{-5}	$C = 4$
Batch $\times$ accum.	8×4	8×4	16	—
Epochs	1	1	3	—
Schedule (warm-up)	cos. (3%)	cos. (3%)	cos. (6%)	—
Seeds	42, 2, 3, 4	42, 2	42, 2, 3	—

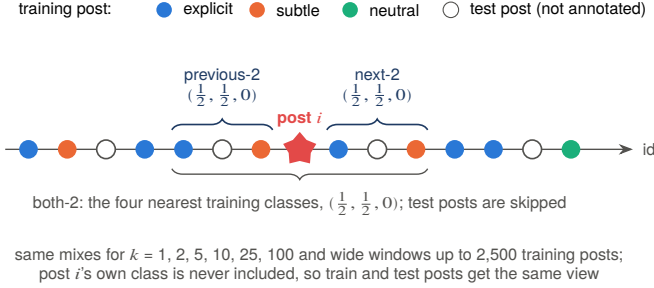


Fig. 5. **Construction of the sequential neighbourhood features for post i** (star). Each window takes the class mix of the k nearest training posts before and/or after i , skips unannotated test posts and never includes i itself, so training and test posts are treated identically.

Seed runs of the same member are averaged before stacking, giving 12 text columns (4 members $\times$ 3 classes).

D. Leakage-safe design

The system uses only the provided competition data: no external corpora, no translation or augmentation data, and no test annotations. Because target-derived features are a classic source of leakage [27], every feature obeys three rules. (i) Neighbourhood windows end strictly before and start strictly after the post, over training posts only (Fig. 5), so a post’s own class never enters its features; we verified this by perturbing a training post’s class and checking that its own features do not change. (ii) A test post gets exactly the view a training post gets. (iii) Neighbours’ text-model predictions are *not* used: a training neighbour’s OOF prediction comes from a model whose training folds contained the current post and can carry its class, which never happens for a test post.

A detected leak. One candidate feature, the class mix of the post’s entire language run (a maximal block of same-script posts) minus the post’s own class, raised the stacker to 0.97 OOF. All posts in a run share the run total, so “total minus own class” differs between them only by their own class: it is a leave-one-out target encoding. The annotation-noise ceiling made the leak obvious (0.97 is far above what 84% twin agreement allows), and the feature was discarded before any submission.

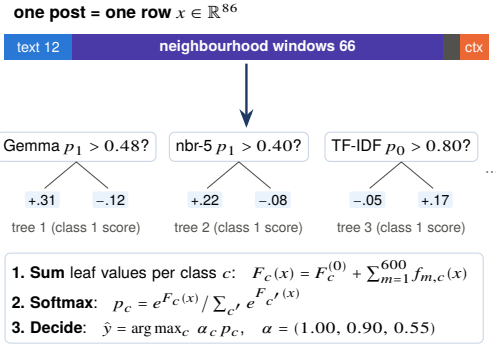


Fig. 6. **How the level-2 stacker reaches a decision** (schematic; thresholds and leaf values are illustrative). Shallow trees are built in sequence, each correcting the errors of its predecessors; their leaf values are summed per class, turned into probabilities and rescaled by the macro-F1 multipliers. Because splits combine several members’ scores with context evidence, the stacker learns *when* to trust which model.

E. Level-2 stacker

The stacker is scikit-learn’s HistGradientBoostingClassifier [21], [25]: 600 boosting rounds (one tree per class per round), learning rate 0.04, 31 leaves, at least 40 samples per leaf, L2 regularisation 1.0, balanced class weights and early stopping on an internal 10% split. It is trained on the same five folds as level 1, so its own OOF predictions stay honest, and it is bagged over five seeds; the test prediction is the mean of the 25 fold-seed models. Fig. 6 shows how a single row becomes a prediction: each tree routes the row through a few splits on model probabilities and context features and adds a small value to one class score; the summed scores are passed through a softmax.

F. Calibration for macro F1

Balanced class weights make the stacker over-predict the rare neutral class. Since F1-optimal decisions differ from the argmax [28], we predict $\hat{y} = \arg \max_c \alpha_c p_c$ and choose α by coordinate ascent over a log-spaced grid of 65 values in $[e^{-1.6}, e^{1.6}]$ (four rounds) to maximise OOF macro F1. For the final stacker, $\alpha = (1, 0.90, 0.55)$: neutral is roughly halved. To check that this does not overfit, we also fit α on folds 0–2 only and score folds 3–4 (“held-out”).

G. Verbatim override

The 272 test posts whose cleaned text occurs in the training data with a single class take that class (34 predictions change; +0.0005 in a leave-one-out simulation on train).

V. EXPERIMENTAL SETUP

Implementation. Every run is tracked in a run registry, and scripts verify output shapes, folds and scores before a run is admitted to the stack. Software: PyTorch 2.12, Transformers 5.6, PEFT 0.19, bitsandbytes 0.50, scikit-learn 1.9.

Reported scores. *OOF* is the macro F1 of the argmax of the 47,817 out-of-fold predictions; *tuned* applies the per-class multipliers fitted on OOF; *held-out* fits them on folds 0–2 and scores folds 3–4; *public* and *private* are leaderboard

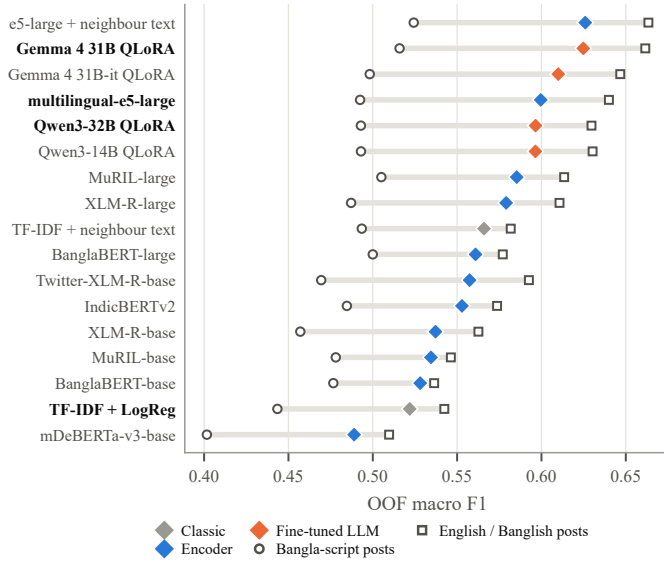


Fig. 7. **Comparison of 17 fine-tuned level-1 models**, ranked by OOF macro F1 (diamonds; colour = model family). Circles and squares give the score on Bangla-script and on English/Banglish posts; bold names are the final stack members. Fine-tuned Gemma 4 31B is the strongest single model, and every model is weaker on Bangla.

scores. All model selection used OOF and held-out scores only. Ablations re-run the full 5-fold stacker for three GBDT seeds per configuration (mean $\pm$ standard deviation).

VI. RESULTS

A. Level-1 models

Table III and Fig. 7 compare all fine-tuned level-1 models. Four observations shaped the system:

- **Model family mattered more than size.** Gemma 4 31B reaches 0.625, about 0.03 above Qwen3-32B (0.596), which is no better than Qwen3-14B.
- **Fine-tuning beats prompting.** The same Gemma checkpoint scores 0.553 as a 9-shot judge (probabilities from class-token log-probabilities, vLLM) and 0.625 fine-tuned; a Qwen3-32B judge reaches 0.499.
- **A strong encoder is cost-effective.** multilingual-e5-large reaches 0.600 in about 12 minutes per fold, against about 3.6 hours per fold for Gemma.
- **Every model is weaker on Bangla** (Fig. 7), by 0.06–0.15 macro F1; Gemma scores 0.516 on Bangla against 0.662 on English/Banglish. Reading neighbouring posts’ text helps text models (e5-large 0.600 $\rightarrow$ 0.626).

Fig. 8 shows typical training curves. Gemma’s and Qwen’s losses fall quickly and flatten within one epoch; e5-large’s training loss drops at every epoch boundary while its validation F1 stays flat, so longer training would mostly memorise training posts.

B. Main results

Table IV summarises the main results. Our selected submission (stack v9 with a 5-seed bag) scored 0.7554 on the private

leaderboard, 0.0003 above its OOF estimate. Re-running it from the saved level-1 predictions for this paper reproduces both numbers exactly (0.7551 OOF, 0.7547 held-out); all analyses below use this run. Calibration lifts its OOF from 0.7456 to 0.7551 (+0.0095), and the held-out estimate (0.7547) confirms that the multipliers do not overfit. Per class it reaches F1 = 0.814 (explicit), 0.710 (subtle) and 0.742 (neutral), with accuracy 0.765.

C. Error analysis

Most errors lie on the explicit–subtle border (Fig. 9): 4,233 explicit posts are predicted subtle and 3,928 subtle posts explicit. Neutral posts are confused mainly with subtle ones (1,198), while explicit and neutral are rarely mixed (345 and 287). By script, the stack scores 0.7245 on Bangla-script posts and 0.7610 on English and Banglish posts. Fig. 10 breaks the score down by id region: Bangla-heavy regions are the hardest (down to 0.68), while the best region reaches 0.89, where neighbouring posts are unusually consistent in class.

D. Case studies

VII. ABLATION STUDY

A. Feature groups and members

Table VI and Fig. 11 remove one input at a time from the final stacker.

Text probabilities are essential. Removing all twelve text columns costs -0.0877, while the script and length context adds little (-0.0019).

Gemma is the only individually important member. Removing Gemma costs -0.0105. Removing e5-large (+0.0008), Qwen3-32B (+0.0007) or TF-IDF (+0.0023) does not hurt: the changes are within or just above the seed noise. Given Gemma and the other features, their probabilities add mostly correlated noise for the trees to split on. A stacker with Gemma as its *only* text member scores 0.7582 (+0.0047), better than the full model and far cheaper. These leaner variants were found after the competition with this ablation; choosing them on the same folds would carry the same small selection optimism discussed in Sec. IX, so we report them as findings rather than as our result.

B. Member selection

Table VII shows how the member set was chosen. Adding every available text model *lowered* the stack (0.7199 with all 12 against 0.7231 with 9), and the large LLM fine-tunes lifted it by almost 0.03. We kept four diverse members; the ablation above shows that, given the id features, Gemma alone carries most of the text signal.

C. Stacker family, variance and late variants

Table VIII compares stacker families and the variants explored in the final hours. All tree ensembles tie within noise, while a linear stacker loses about 0.025: the interactions between model confidence and neighbourhood evidence matter. Re-running the same stacker with another seed or scikit-learn build moved the held-out score by up to 0.004, more than most

TABLE III

ALL FINE-TUNED LEVEL-1 MODELS (5-FOLD OOF MACRO F1; SEED RUNS AVERAGED BEFORE SCORING). “TUNED” APPLIES PER-CLASS MULTIPLIERS FITTED ON OOF. $F1_c$ IS THE PER-CLASS F1 OF THE ARGMAX; THE LAST COLUMN SPLITS OOF MACRO F1 BY WHETHER THE POST CONTAINS BANGLA SCRIPT (OTHER = ENGLISH AND BANGLISH). * = MEMBER OF THE FINAL STACK; BOLD = BEST IN COLUMN.

Model	Seeds	OOF	Tuned	$F1_0$	$F1_1$	$F1_2$	Bn / Other
<i>Classical</i>							
TF-IDF + LogReg*	1	0.522	0.525	0.647	0.498	0.420	0.443 / 0.542
TF-IDF + neighbour text	1	0.566	0.567	0.685	0.543	0.470	0.493 / 0.582
<i>Encoders (fine-tuned)</i>							
mDeBERTa-v3-base	1	0.489	0.529	0.664	0.382	0.420	0.402 / 0.510
BanglaBERT-base	1	0.528	0.553	0.656	0.473	0.455	0.477 / 0.536
MuRIL-base	1	0.535	0.562	0.672	0.456	0.476	0.478 / 0.546
XLM-R-base	1	0.537	0.564	0.668	0.473	0.471	0.457 / 0.563
IndicBERTv2	1	0.553	0.574	0.670	0.505	0.484	0.485 / 0.574
Twitter-XLM-R-base	1	0.557	0.578	0.667	0.507	0.497	0.469 / 0.593
BanglaBERT-large	2	0.561	0.570	0.661	0.531	0.490	0.500 / 0.577
XLM-R-large	3	0.579	0.590	0.689	0.521	0.527	0.487 / 0.611
MuRIL-large	3	0.585	0.594	0.688	0.545	0.523	0.505 / 0.613
multilingual-e5-large*	3	0.600	0.604	0.700	0.551	0.548	0.492 / 0.640
e5-large + neighbour text	3	0.626	0.631	0.714	0.572	0.592	0.524 / 0.663
<i>Decoder LLMs (QLoRA)</i>							
Qwen3-14B	2	0.596	0.613	0.701	0.533	0.556	0.493 / 0.630
Qwen3-32B*	2	0.596	0.611	0.698	0.538	0.554	0.493 / 0.630
Gemma 4 31B-it	1	0.610	0.620	0.707	0.549	0.574	0.498 / 0.647
Gemma 4 31B*	4	0.625	0.636	0.717	0.563	0.594	0.516 / 0.662

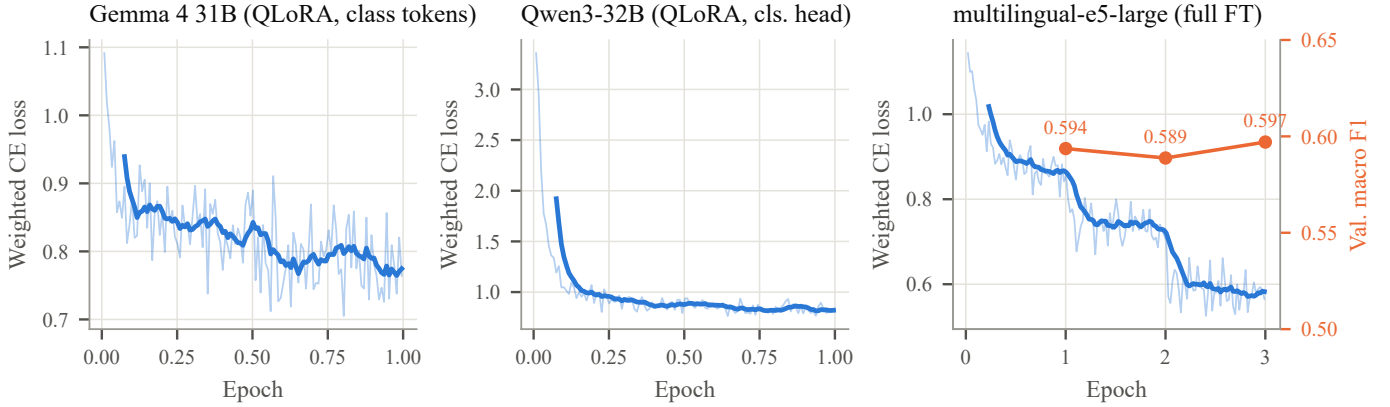


Fig. 8. **Training dynamics on fold 0** (thin: logged loss; thick: moving average). Both QLoRA LLMs converge within a single epoch; Qwen3-32B’s logged loss is divided by its four gradient-accumulation steps. For e5-large (right), training loss keeps falling across epochs while validation macro F1 (orange) stays flat, the signature of memorisation, which is why we stop at three epochs.

TABLE IV

MAIN RESULTS (MACRO F1). THE SELECTED SYSTEM’S PRIVATE SCORE LIES WITHIN 0.0003 OF ITS CROSS-VALIDATION ESTIMATE. LEADERBOARD SCORES FROM THE COMPETITION’S SUBMISSION PAGE.

System	OOF tuned	Held-out	Public	Private
TF-IDF + LogReg	0.5249	–	0.54629	–
Gemma 4 31B (4 seeds)	0.636	–	–	–
Stack v9, 5-seed bag (selected)	0.7551	0.7547	0.74088	0.75543

feature ideas, which is why the selected submission averages five seeds per fold.

D. Negative results

Table IX lists ideas that did not help. Pseudo-annotations [26] (confident test predictions reused as training data), generated fold-blind so that the OOF estimate

stays honest, improved single encoders by 0.01–0.03 but not the stack: e5-large trained on pseudo-annotations agrees with its own pseudo-annotations on 87.6% of those test posts against 77.4% for plain e5-large, so the stacker, which learns trust from OOF rows where no such memorisation exists, would over-trust the member on test.

VIII. GENERALISATION AND LEADERBOARD ANALYSIS

Our selected submission finished second on the private leaderboard (Table XI), up from third on the public one, and its private score landed within 0.0003 of the cross-validation estimate we selected it by. This was not luck: Table X and Fig. 12 show that *every* stack submission scored within 0.004 of its cross-validation estimate on the private split, while public scores sat about 0.015 below it. Selecting by CV was the right call: the submission with the best public score (0.74302,

TABLE V

CASE STUDIES: REAL TRAINING POSTS (MILD EXAMPLES) WITH EACH MEMBER’S OUT-OF-FOLD PREDICTION AS CLASS PROBABILITY, THE CLASS MIX OF THE 5 PREVIOUS AND 5 NEXT TRAINING POSTS, AND THE CALIBRATED STACKER OUTPUT (STACK v9, 5-SEED BAG). * = WRONG. THE STACKER HAS NO FIXED VOTING RULE: IT LEARNS FROM 47,817 OOF ROWS WHEN TO TRUST WHICH SOURCE.

Post	True	Gemma	Qwen	e5-large	TF-IDF	Neighbours	Stack
(a) <i>All sources agree</i>							
CNN is praying that this doesn’t work	1	1 0.72	1 0.57	1 0.93	1 0.54	1 0.60	1 0.94
(b) <i>Gemma and Qwen disagree: the stacker follows Gemma, backed by the neighbours</i>							
Jakes head is so big, joshua wont be able to miss it.	1	1 0.79	0 0.53*	0 0.72*	1 0.44	1 0.60	1 0.88
(c) <i>All text models are wrong: neighbourhood classes decide (Banglish)</i>							
Oder supporter ra blindly leader er kotha fol-low kore.	0	2 0.42*	2 0.41*	1 0.66*	2 0.58*	0 1.00	0 0.99
(d) <i>TF-IDF and the neighbours outvote the LLMs (Bangla: “Quit the empty talk and take to the streets, uncle”)</i>							
ফাকা বুলা বাদ দিয়ে রাস্তায় নামেন কারু	2	1 0.47*	0 0.49*	0 0.46*	2 0.46	2 0.70	2 0.74
(e) <i>Failure: every source says explicit, the gold class is subtle</i>							
Sit down old man	1	0 0.53*	0 0.52*	0 0.56*	0 0.70*	0 0.40*	0 0.53*

TABLE VI

STACKER ABLATION ON THE FEATURE SET OF THE SELECTED SUBMISSION (STACK v9, 86 COLUMNS). EACH ROW RE-RUNS THE 5-FOLD STACKER FOR THREE GBDT SEEDS (ONE MODEL PER FOLD AND SEED); OOF TUNED IS THE MEAN $\pm$ STANDARD DEVIATION, Δ IS THE MEAN PAIRED DIFFERENCE TO THE FULL STACKER (RED: BELOW -0.002 , THE SEED-NOISE LEVEL; GREEN: ABOVE $+0.002$), HELD-OUT FITS THE MULTIPLIERS ON FOLDS 0–2 AND SCORES FOLDS 3–4. BANGLA/OTHER: TUNED OOF MACRO F1 ON BANGLA-SCRIPT AND ON ENGLISH/BANGLISH POSTS.

Stacker input	Cols	OOF tuned	Δ	Held-out	Bangla	Other
Full stacker (stack v9, selected)	86	0.7536 $\pm$ 0.0001	—	0.7541	0.720	0.760
– all text probabilities	74	0.6659 $\pm$ 0.0017	-0.0877	0.6702	0.668	0.652
– script / length context	81	0.7517 $\pm$ 0.0006	-0.0019	0.7515	0.713	0.759
– Gemma 4 31B	83	0.7431 $\pm$ 0.0007	-0.0105	0.7440	0.706	0.751
– multilingual-e5-large	83	0.7544 $\pm$ 0.0003	+0.0008	0.7563	0.718	0.762
– Qwen3-32B	83	0.7543 $\pm$ 0.0006	+0.0007	0.7552	0.716	0.762
– TF-IDF	83	0.7559 $\pm$ 0.0007	+0.0023	0.7565	0.721	0.763
Gemma as the only text member	77	0.7582 $\pm$ 0.0004	+0.0047	0.7587	0.725	0.765

True	Explicit	0.81 19,178	0.18 4,233	0.01 345
	Subtle	0.22 3,928	0.72 12,991	0.07 1,244
	Neutral	0.05 287	0.20 1,198	0.75 4,413
		Explicit	Subtle	Neutral
		Predicted		

Fig. 9. **Confusion matrix of the selected system** on all 47,817 out-of-fold predictions (row-normalised rates; counts below). Most confusions lie between explicit and subtle toxicity; explicit and neutral posts are almost never mixed up.

third on the public leaderboard) has the lowest private score among the stacks, while the selected v9 bag matched its CV to within 0.0003. We tested three explanations for the public gap. *Pseudo-annotation memorisation*: stack v9 without pseudo-annotated members scored the same as v8 with them.

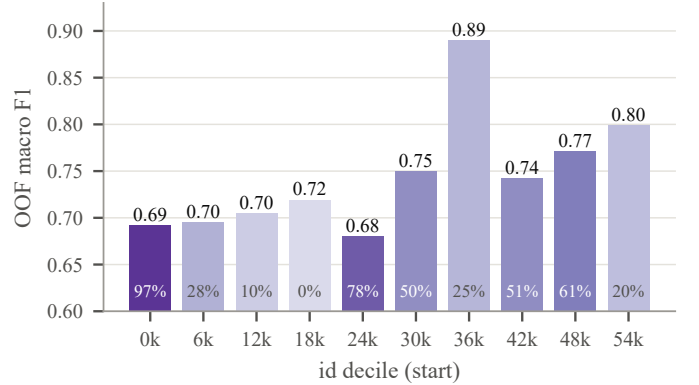


Fig. 10. **Macro F1 of the selected system in each tenth of the id range.** Bar shade and the number in each bar give the share of Bangla-script posts: the Bangla-heavy regions are the hardest.

Calibration shift: test and OOF predictions have the same class mix and confidence. *A non-random public slice*: we scored the OOF predictions of training posts inside every contiguous id window that holds 40% of the test posts.

Contiguous windows reproduce all three public scores at once (RMSE $\leq$ 0.007, Table XII); random subsets rarely do (fewer than 1 in 200 match as well). The window score

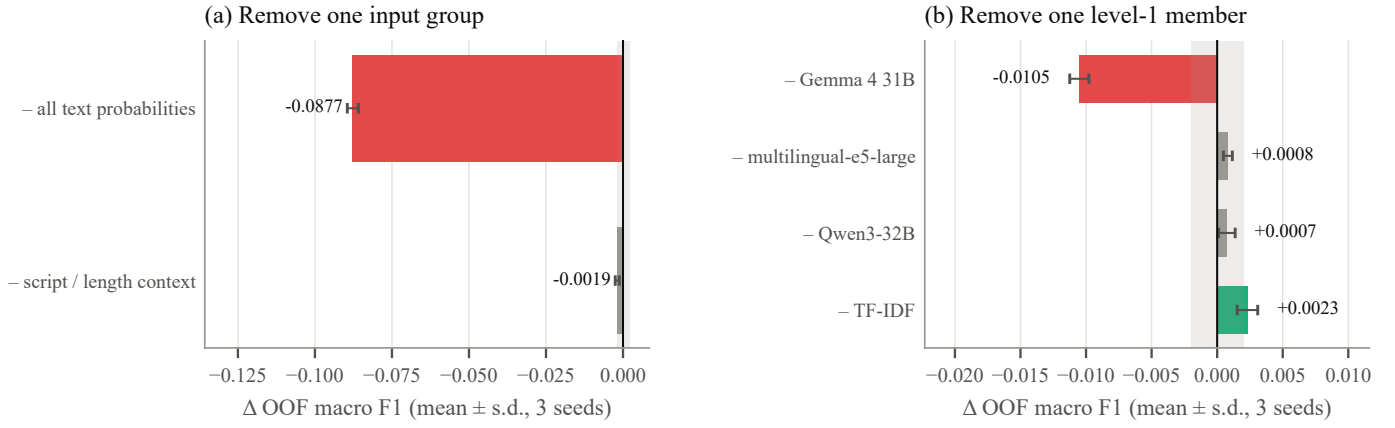


Fig. 11. **Leave-one-out ablation of the selected stacker:** change in OOF macro F1 (mean $\pm$ s.d. over three seeds) when (a) one input group or (b) one level-1 member is removed. The grey band marks the ± 0.002 seed noise; red and green bars are losses and gains beyond it. Gemma is the only member whose removal hurts.

TABLE VII

MORE MEMBERS ARE NOT BETTER: DEVELOPMENT OF THE MEMBER SET (ONE STACKER SEED, ID FEATURES OF THE TIME). GBDT SPENDS SPLITS ON SMALL, NOISY DIFFERENCES BETWEEN MANY CORRELATED PROBABILITY COLUMNS.

Member set (+ id features)	OOF tuned	Held-out
v6: 9 encoder members	0.7231	0.7266
all 12 text models	0.7199	0.7216
v7: best 5 members (no LLMs)	0.7253	0.7289
11 members including Gemma	0.7501	0.7507
Gemma 4 31B + TF-IDF	0.7533	0.7546
v8: Gemma + e5-large + Qwen3-32B + TF-IDF	0.7537	0.7546

TABLE VIII

STACKER CHOICE AND LATE VARIANTS ON THE v9 MEMBER SET (ONE SEED UNLESS STATED). MOST DIFFERENCES FALL INSIDE THE STACKER’S SEED NOISE (± 0.003).

Variant	OOF tuned	Held-out
HistGBDT baseline (<i>stack.py</i>)	0.7555	0.7580
LightGBM	0.7567	0.7573
CatBoost	0.7547	0.7551
XGBoost	0.7539	0.7561
Logistic regression (spline features)	0.7304	0.7331
HistGBDT, 5 seeds averaged	0.7559	0.7574
Mean of six stackers (“grand6”)	0.7576	0.7594
+ classes of the 20 most similar texts	0.7522	0.7554
+ context-aware e5 added	0.7527	0.7531
context-aware e5 instead of e5	0.7538	0.7544
+ positional neighbour classes $\pm 1..6$, text similarity	0.7492	0.7504
Two-stage cascade	0.7496	0.7492
Extra id windows ($k = 3, 50, 5,000$)	0.7572	0.7571
Slower GBDT	0.7527	0.7540
Heavier regularisation	0.7508	0.7524

varies from 0.709 to 0.812 (Fig. 13), far outside the random-subset band. The public part of the test set is therefore most likely a contiguous low-to-middle id range, our hardest region. Attempts to adapt to it (region-specific multipliers or a stacker weighted towards low ids) lost 0.005–0.018 on held-out folds, so we kept selecting by CV, which the private results confirm.

TABLE IX

IDEAS THAT DID NOT IMPROVE THE STACK (HELD-OUT MACRO F1 AGAINST THE STACK OF THE TIME).

Idea	Effect
Pseudo-annotated members in the stack	same public score; memorisation
Classes of the most similar training texts	−0.003
Positional neighbour classes + text similarity	−0.008
HMM / sequence-smoothing features	−0.002
Bangla specialists (BanglaBERT-L, MuRIL-L) added	−0.003
Two-stage cascade	−0.009
Region-specific multipliers or stackers	−0.005 to −0.018
Neighbours’ text-model probabilities	+0.000, leak risk
Language-run class mix minus own class	0.97 OOF: leakage

TABLE X

ALL STACK SUBMISSIONS. “SEL.” MARKS THE TWO SUBMISSIONS SELECTED FOR THE PRIVATE RANKING. *LATE SUBMISSION, AFTER THE DEADLINE.

Submission	OOF	Held-out	Public	Private	Sel.
v8 (pseudo-annotated members)	0.7524	0.7541	0.73843	—	
v9, one stacker seed	0.7529	0.7542	0.73848	0.75634	
v9 + TF-IDF-context, bag-5	0.7546	0.7570	0.74302	0.75189	
v9, bag-5	0.7551	0.7547	0.74088	0.75543	
grand6* (mean of 6 stackers)	0.7576	0.7594	0.74324	0.75583	

IX. DISCUSSION

Why the system generalises. Four design choices kept the private score predictable. (i) Every stacked prediction is out-of-fold, so the stacker learns how far to trust each member only from rows that member never saw. (ii) Every feature is computed identically for training and test posts and never uses a post’s own class. (iii) Calibration was validated on held-out folds before use (0.7547 held-out against 0.7551 in-sample). (iv) All selection was done on 47,817 cross-validated rows rather than on the roughly 4,800-post public split, which Sec. VIII shows to be a harder, contiguous slice.

Language disparity. Every model is weaker on Bangla-

TABLE XI
FINAL PRIVATE LEADERBOARD, TOP FOUR TEAMS (MACRO F1 ON THE HIDDEN 60% OF THE TEST SET).

Rank	Team	Private macro F1
1	FatWiz	0.77514
2	IUT BakeNekos (ours)	0.75543
3	CUET_AL_Masaar_56	0.75113
4	event_horizon	0.74842

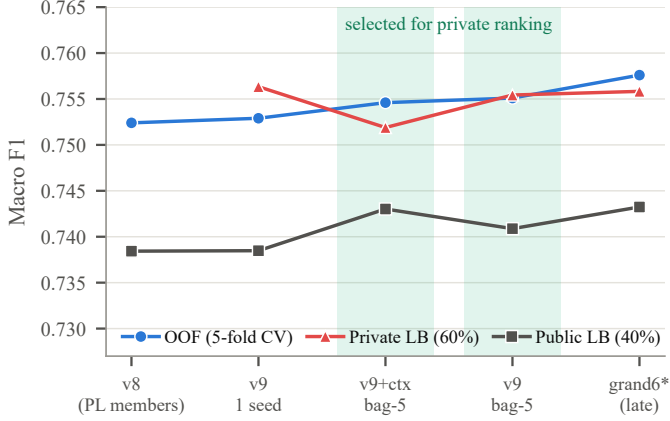


Fig. 12. **Cross-validation tracks the private leaderboard.** OOF, public and private macro F1 of every stack submission: private scores stay within 0.004 of cross-validation, while public scores sit about 0.015 below it. Green bands: the two submissions selected for the private ranking.

script posts: 0.7245 against 0.7610 for the stack, and 0.12–0.15 lower for the strongest text-only models. Automatic removal decisions on Bangla content would therefore carry more errors; a responsible deployment would route low-confidence Bangla predictions to human review and monitor error rates per language [3], [4].

Ambiguity of subtle toxicity. Near-identical posts disagree on their class about 16% of the time, and most errors fall between explicit and subtle or subtle and neutral. Class-1 predictions are better used to inform review than to trigger automatic penalties; class weighting and macro-F1 calibration keep the neutral class from being absorbed, which protects ordinary speech from being flagged by default.

Limitations. The results from one dataset. Several choices (member sets, windows) were selected among a handful of candidates on the same folds; we estimate the resulting optimism at about 0.002 and mitigate it with the held-out protocol. Gradient boosting differs slightly across scikit-learn builds (about 98% agreement on test predictions), and neural-network training is not bit-for-bit deterministic. Context-aware Gemma, the most promising unexplored model, could not be run within the competition window.

X. CONCLUSION

We presented a leakage-safe, two-level system for three-class harmful-content classification in Bangla, English and Banglish. Fine-tuned LLMs, Gemma 4 31B in particular, were the strongest text models; combined with sequential

TABLE XII
MATCHING THREE PUBLIC SCORES WITH OOF SCORES ON CANDIDATE SLICES OF THE TRAINING DATA.

Rows scored	Stack v9	TF-IDF	TF-IDF tuned	RMSE
Public leaderboard	0.7385	0.5409	0.5463	–
ids 0–24,428	0.7275	0.5396	0.5421	0.0068
ids 14,551–38,901	0.7437	0.5357	0.5376	0.0066
200 random 40% subsets				best 0.0077

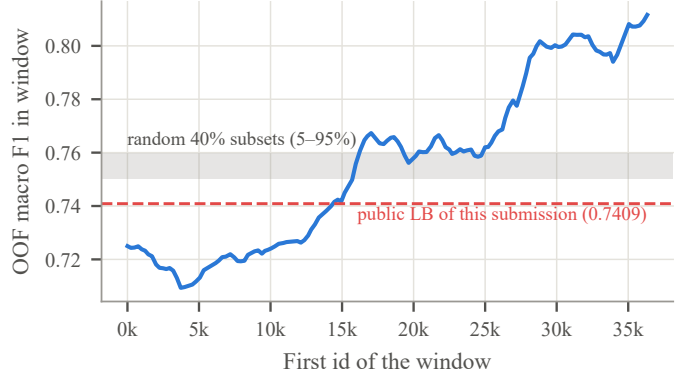


Fig. 13. **Why the public leaderboard misled.** OOF macro F1 of the selected system inside every contiguous id window holding 40% of the test posts, against the window’s first id. Scores swing from 0.709 to 0.812, far outside the band of random 40% subsets (grey, 5th–95th percentile); the low-id windows reproduce the public scores, marking the public split as a harder, contiguous slice.

neighbourhood features in a bagged gradient-boosting stacker, they reach 0.755 macro F1 and placed second on the private leaderboard. Above all, the system generalised as predicted: cross-validation matched the private score to within 0.0003, while the public leaderboard, a contiguous and harder slice, did not. For practitioners, our results argue for two habits: build the validation contract first, and treat any score above the annotation-noise ceiling as a bug.

ACKNOWLEDGMENT

We thank the HerWILL and UAP organisers for the datathon, Kaggle, and our collaborator who ran the large-model experiments.

REFERENCES

- [1] T. Davidson, D. Warmley, M. Macy, and I. Weber, “Automated hate speech detection and the problem of offensive language,” in *Proc. ICWSM*, 2017.
- [2] M. ElSherief, C. Ziem, D. Muchlinski, V. Anupindi, J. Seybolt, M. De Choudhury, and D. Yang, “Latent hatred: A benchmark for understanding implicit hate speech,” in *Proc. EMNLP*, 2021.
- [3] M. Sap, D. Card, S. Gabriel, Y. Choi, and N. A. Smith, “The risk of racial bias in hate speech detection,” in *Proc. ACL*, 2019.
- [4] D. Borkan, L. Dixon, J. Sorensen, N. Thain, and L. Vasserman, “Nuanced metrics for measuring unintended bias with real data for text classification,” in *Companion Proc. WWW*, 2019.
- [5] M. R. Karim *et al.*, “DeepHateExplainer: Explainable hate speech detection in under-resourced Bengali language,” in *Proc. IEEE DSAA*, 2021.
- [6] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, and M. Choudhury, “GLUECoS: An evaluation benchmark for code-switched NLP,” in *Proc. ACL*, 2020.

- [7] A. Bhattacharjee *et al.*, “BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla,” in *Findings of NAACL*, 2022.
- [8] S. Khanuja *et al.*, “MuRIL: Multilingual representations for Indian languages,” arXiv:2103.10730, 2021.
- [9] S. Doddapaneni *et al.*, “Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages,” in *Proc. ACL*, 2023.
- [10] A. Conneau *et al.*, “Unsupervised cross-lingual representation learning at scale,” in *Proc. ACL*, 2020.
- [11] F. Barbieri, L. Espinosa Anke, and J. Camacho-Collados, “XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond,” in *Proc. LREC*, 2022.
- [12] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” in *Proc. ICLR*, 2023.
- [13] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 text embeddings: A technical report,” arXiv:2402.05672, 2024.
- [14] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *Proc. ICLR*, 2022.
- [15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Proc. NeurIPS*, 2023.
- [16] Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.
- [17] Gemma Team, Google DeepMind, “Gemma 4 open models,” model cards google/gemma-4-31B and google/gemma-4-31B-it, 2026.
- [18] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [19] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, 2020.
- [20] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [21] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [22] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Proc. NeurIPS*, 2017.
- [23] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased boosting with categorical features,” in *Proc. NeurIPS*, 2018.
- [24] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. KDD*, 2016.
- [25] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [26] D.-H. Lee, “Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks,” in *ICML Workshop on Challenges in Representation Learning*, 2013.
- [27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.
- [28] Z. C. Lipton, C. Elkan, and B. Naryanaswamy, “Optimal thresholding of classifiers to maximize F1 measure,” in *Proc. ECML PKDD*, 2014.